

1.Introducción

El reporte actual examina cómo funcionan y cuán eficientes son dos algoritmos de ordenamiento básicos: Quicksort, que usa comparaciones, y Dial Sort (Dsort), una forma de ordenamiento por distribución (Radix Sort) hecha para cadenas de texto. El objetivo es ver cuál funciona mejor en tiempo de ejecución y cantidad de datos que puede manejar cuando hay muchos datos.

2.Descripción del algoritmo

1) *Quicksort (Algoritmo de comparación)*

El algoritmo Quicksort utiliza una técnica muy útil que se llama “divide y vencerás”. Esto significa que selecciona un elemento especial dentro del arreglo, al que llamamos pivote. Luego, lo que hace es ordenar el arreglo de tal manera que todos los elementos que son menores que el pivote quedan a la izquierda de este, y todos los elementos que son mayores que el pivote quedan a la derecha. Esto facilita mucho el proceso de ordenación.

- **Complejidad Temporal:**
 - Promedio: $O(n \log n)$
 - Peor caso: $O(n^2)$ (cuando el pivote es el elemento más pequeño o más grande repetidamente).
- **Complejidad Espacial:** $O(\log n)$ debido a la pila de recursión.
- **Características:** No es un algoritmo estable, lo que significa que elementos con la misma clave pueden cambiar su orden relativo.

2) *Dial Sort / Dsort (Algoritmo de Distribución)*

Esta forma de Dial Sort funciona como un LSD Radix Sort, que es un tipo de ordenación. En lugar de comparar las palabras completas, divide las cadenas en grupos llamados “buckets” según el valor ASCII de cada carácter. Lo hace desde el último carácter de la palabra hasta el primero.

- **Complejidad Temporal:** $O(n \cdot k)$, donde n es el número de elementos y k es la longitud promedio de las cadenas.
- **Complejidad Espacial:** $O(n + \text{alfabeto})$, ya que requiere un vector auxiliar (output) y un arreglo de conteo.
- **Características:** Es un algoritmo estable y su rendimiento no depende de comparaciones directas, sino de la distribución de los datos

3. Metodología de pruebas

Para garantizar la validez de los resultados, se implementó el siguiente protocolo:

1. **Dataset:** Se utilizó un archivo dataset.txt cargado dinámicamente en memoria.
2. **Aleatorización:** Antes de cada iteración, se aplicó un `std::shuffle` al conjunto de datos para evitar sesgos por ordenamientos previos.
3. **Métricas:** Se realizaron 5 iteraciones por algoritmo para calcular:
 - **Media:** Tiempo promedio de ejecución en milisegundos.
 - **Desviación Estándar:** Variabilidad de los tiempos respecto a la media.
 - **Throughput:** Cantidad de registros procesados por segundo ($n/\text{tiempo en seg}$)

4. Implementación

El sistema se dividió en tres componentes modulares:

- `practicaFinal.h`: Definición de prototipos y estructuras.
- `practicaFinal.cpp`: Implementación de la lógica de ordenamiento y funciones de simulación.
- `main.cpp`: Orquestador de la simulación visual y el motor de benchmarking.

Se agregó un simulador visual. Este simulador permite ver cómo funcionan los algoritmos de manera lógica con un ejemplo pequeño. Así se puede comprobar si el algoritmo es estable y cómo se divide en el caso de Quicksort. También se puede comparar con la forma en que se reorganizan los elementos según su posición en Dial Sort.

5. Análisis de resultados esperados

Basado en la naturaleza de los algoritmos sobre datos de tipo texto:

1. **Eficiencia:** Se espera que **Dial Sort** supere a Quicksort en grandes volúmenes de datos donde las palabras son relativamente cortas, ya que el costo de comparación de `std::string` en Quicksort es mayor que el costo de indexación de caracteres en Dial Sort.

2. **Uso de Recursos:** Quicksort es más eficiente en uso de memoria ("in-place"), mientras que Dial Sort requiere memoria adicional proporcional al tamaño del dataset para sus estructuras auxiliares.
3. **Estabilidad:** Dial Sort conserva el orden original de elementos idénticos, lo cual es una ventaja crítica en bases de datos con múltiples criterios de ordenamiento.

6. Conclusiones

La comparación técnica muestra que, para procesar muchas cadenas de caracteres, los algoritmos de distribución como Dial Sort son mejores. Esto se debe a que su complejidad es lineal respecto al número de elementos. Sin embargo, Quicksort es la opción que más se prefiere en sistemas que tienen limitaciones de memoria RAM.